

Algoritmos de Optimización para Redes Neuronales

Este documento detalla el funcionamiento matemático y conceptual de los algoritmos de optimización más utilizados en el entrenamiento de redes neuronales profundas.

Autor: Dr. Aboud Barsekh Onji **Institución:** IPN - Universidad Anáhuac México

1. Descenso de Gradiente Estocástico con Momento (SGDM)

El **Stochastic Gradient Descent (SGD)** es el algoritmo base, donde los pesos se actualizan en la dirección opuesta al gradiente de la función de pérdida. Sin embargo, SGD puede oscilar mucho y converger lentamente. **SGDM** (con Momentum) mejora esto acumulando un promedio móvil de los gradientes pasados, lo que ayuda a “tomar impulso” en direcciones constantes y amortiguar oscilaciones.

Ecuaciones

Sea θ_t el vector de parámetros (pesos) en la iteración t , α la tasa de aprendizaje (*learning rate*), y $\nabla L(\theta_t)$ el gradiente de la función de pérdida.

1. Cálculo del Momento (Velocidad):

$$v_{t+1} = \beta \cdot v_t + (1 - \beta) \cdot \nabla L(\theta_t)$$

- v_t : Vector de velocidad (acumulador de gradientes).
- β : Factor de momento (generalmente 0.9). Controla cuánto de la historia pasada se retiene.

2. Actualización de Parámetros:

$$\theta_{t+1} = \theta_t - \alpha \cdot v_{t+1}$$

Características

- **Ventajas:** Acelera la convergencia en áreas con curvatura constante y reduce el ruido de los gradientes estocásticos.
 - **Desventajas:** La tasa de aprendizaje α es fija para todos los parámetros, lo que puede ser ineficiente si algunos parámetros necesitan pasos grandes y otros pequeños.
-

2. AdaGrad (Adaptive Gradient Algorithm)

AdaGrad adapta la tasa de aprendizaje para cada parámetro individualmente. Realiza actualizaciones más grandes para parámetros poco frecuentes (gradientes esparcidos) y actualizaciones más pequeñas para parámetros frecuentes.

Ecuaciones

Sea $g_{t,i} = \nabla_{\theta} L(\theta_t)_i$ el gradiente del parámetro i -ésimo en el paso t .

1. Acumulación de Gradientes al Cuadrado:

$$G_{t,i} = G_{t-1,i} + g_{t,i}^2$$

- $G_{t,i}$: Suma acumulada de los cuadrados de los gradientes históricos para el parámetro i .

2. Actualización de Parámetros:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\alpha}{\sqrt{G_{t,i}} + \epsilon} \cdot g_{t,i}$$

- ϵ : Término pequeño (aprox 10^{-8}) para evitar división por cero.

Características

- **Ventajas:** Elimina la necesidad de ajustar manualmente la tasa de aprendizaje. Muy bueno para datos dispersos (NLP).
- **Desventajas:** $G_{t,i}$ crece monótonamente sin límite. Esto hace que la tasa de aprendizaje efectiva $\frac{\alpha}{\sqrt{G_{t,i}}}$ decaiga a cero prematuramente, deteniendo el aprendizaje en redes profundas.

3. RMSProp (Root Mean Square Propagation)

RMSProp es una modificación de AdaGrad diseñada para resolver el problema de la disminución radical de la tasa de aprendizaje. En lugar de acumular *todos* los gradientes cuadrados históricos, utiliza un promedio móvil exponencial.

Ecuaciones

1. Promedio Móvil de Gradientes al Cuadrado:

$$E[g^2]_t = \gamma \cdot E[g^2]_{t-1} + (1 - \gamma) \cdot g_t^2$$

- $E[g^2]_t$: Promedio exponencial del cuadrado de los gradientes.
- γ : Factor de decaimiento (típicamente 0.9).

2. Actualización de Parámetros:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t} + \epsilon} \cdot g_t$$

Características

- **Ventajas:** Funciona bien en entornos no estacionarios (como redes neuronales profundas) porque olvida la historia muy lejana.
 - **Uso:** Fue el estándar para entrenar RNNs y LSTMs antes de la popularización de Adam.
-

4. Adam (Adaptive Moment Estimation)

Adam combina las mejores ideas de **Momentum** (acumular el promedio de gradientes) y **RMSProp** (acumular el promedio de gradientes al cuadrado). Calcula tasas de aprendizaje adaptativas para cada parámetro.

Ecuaciones

Calcula dos momentos: 1. **Primer Momento (Media - similar a Momentum):**

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

2. **Segundo Momento (Varianza no centrada - similar a RMSProp):**

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

3. **Corrección de Sesgo (Bias Correction):** Como m_0 y v_0 se inicializan en 0, están sesgados hacia 0 al inicio. Se corrige así:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. **Actualización de Parámetros:**

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

Hiperparámetros Típicos

- α (Learning Rate): 0.001
- β_1 (Decaimiento primer momento): 0.9
- β_2 (Decaimiento segundo momento): 0.999
- ϵ : 10^{-8}

Características

- **Ventajas:** Combina la velocidad de convergencia de Momentum con la capacidad de adaptación de RMSProp. Es robusto a la elección de hiperparámetros y requiere poca memoria.
- **Estado Actual:** Es el algoritmo por defecto (“State of the Art”) para la mayoría de las tareas de Deep Learning en la actualidad.

Resumen Comparativo

Algoritmo	Concepto Clave	Principal Ventaja	Principal Desventaja
SGDM	Gradiente + Inercia	Rápido y estable en valles.	Learning rate fijo para todos los parámetros.
AdaGrad	Learning rate adaptativo (historial completo)	Bueno para datos dispersos (Sparse).	El learning rate tiende a cero muy rápido.
RMSProp	Learning rate adaptativo (ventana móvil)	Resuelve el problema de AdaGrad en redes profundas.	A veces oscila más que Adam.
Adam	Momentum + RMSProp	Rápido, estable y requiere poco ajuste (“funciona y ya”).	Puede no generalizar tan bien como SGD puro en algunos casos extremos.